Agentic AI Assessment

Compliance Validator Challenge

📋 Frequently Asked Questions (FAQ)

1. Data & Invoice Formats

Q1.1: The document mentions 150 invoices in mixed formats, but I only see 20 invoices in JSON format. Where are the rest?

A: The 150 invoices mentioned is the total evaluation set, which includes:

- 20 invoices provided to you in test_invoices.json (for development and testing)
- Additional invoices held back for final evaluation (hidden test cases)

Build your solution using the 20 provided invoices. Your system should be generalized enough to handle unseen invoices during evaluation.

⚠ **Important:** Do not hardcode logic specific to the 20 test cases — this is a disqualification criterion.

Q1.2: Do we need to implement OCR for PDF/image invoice processing?

A: No. For this assessment, work directly with the JSON test data provided. You do NOT need to implement actual OCR or PDF parsing.

However, your architecture should demonstrate extensibility:

- Design your Extractor Agent with a clean interface that could accept different input sources
- In your architecture document, describe how you would extend it for PDF/image inputs
- Evaluation focuses on validation logic and agent orchestration, not file parsing

Think of it as: "Build for JSON now, design for everything."

Q1.3: Should we make assumptions about working only with JSON format?

A: Yes, for implementation. Your code should process JSON data. But your design should be format-agnostic. Document your extensibility approach in the architecture document.

2. Mock API

Q2.1: The MOCK_API.md only shows partial code. Do we need to build the complete Mock API ourselves?

A: Yes. You need to implement the Mock API yourself based on the provided specifications. The documentation provides:

- API contract (endpoints, request/response formats)

- Expected behavior and error scenarios
- Sample responses and error codes

This is intentional — it tests your ability to:

- Implement a service based on specifications
- Handle realistic API responses (errors, rate limits, edge cases)
- Design proper tool integration for your agents

Q2.2: What URL should we use for the Mock API?

A: Run the Mock API locally on your machine. You define the URL. Typical setup:

```
http://localhost:5000/api/gst/validate-gstin
http://localhost:5000/api/gst/verify-irn
http://localhost:5000/api/tds/check-206ab
```

Document your setup instructions clearly in your README so evaluators can run your solution.

3. LLM API Keys

Q3.1: What LLM API should we use? Will API keys be provided?

A: We want people to use Groq API credentials for this assessment. Details will be shared separately.

Why Groq?

- Fast inference speeds suitable for multi-agent systems
- Access to models like LLaMA 3 and Mixtral
- Cost-effective for development and testing

Q3.2: Can we use OpenAI/Google Gemini instead?

A: Please use Groq API as provided for consistency in evaluation. The Groq API is compatible with OpenAI-style API calls, so migration is straightforward if you've worked with OpenAI before.

Q3.3: What should we do while waiting for API keys?

A: You can proceedZ with most of the development:

- System architecture design
- Agent definitions and workflow design
- Validation logic implementation (rule-based checks)
- Mock API setup and testing
- Local testing with placeholder/mock LLM responses

Many candidates find it useful to build the full pipeline first and integrate actual LLM calls at the end.

4. Business Logic Clarifications

Q4.1: Payment terms are mentioned, but how do we know if a bill is paid or pending? How to apply discounts or late interest?

A: For this assessment:

- Assume all invoices are unpaid/pending unless explicitly stated otherwise
- payment_terms field indicates the allowed payment window (e.g., "Net 30")

Use invoice_date + payment_terms to determine:

- E8 (Aging beyond payment terms): Is the invoice past its due date based on current processing date?
- E9 (Early payment discount eligibility): Is the invoice within the discount window?

You are validating eligibility and flags, not actual payment status. If your agent needs payment status for certain validations, flag it as "requires additional data" in your output — this demonstrates proper handling of incomplete information.

Q4.2: What date should we use as the "current date" for validation?

A: Use the system date when your code runs, or define a configurable "processing date" parameter. Document your approach. For reproducible testing, a configurable date is recommended.

5. Submission & Evaluation

Q5.1: What exactly will be evaluated?

A: Your submission will be evaluated on:

Accuracy (30%): Correct compliance decisions on test set

Architecture (25%): Clean multi-agent design, separation of concerns

Edge Case Handling (20%): Graceful handling of messy/incomplete data

Reasoning Quality (15%): Explainable decisions, confidence scoring

Code Quality (10%): Clean, documented, testable code

Q5.2: Will there be hidden test cases?

A: Yes. The 20 invoices provided are for development. Additional test cases (including edge cases and boundary conditions) will be used during evaluation. Build a generalized solution, not one tuned to the 20 samples.

6. Quick Reference Summary

Topic	Answer
Invoice Format	Work with JSON only; design for extensibility
OCR Implementation	Not required
Mock API	Build yourself; run on localhost

LLM API	Groq API (credentials shared separately)
Test Invoices	20 provided; more in hidden evaluation set
Payment Status	Assume unpaid; validate eligibility only

Still have questions?
Regulations: DMAgentic.ArmyPanel@datamatics.com
Technical: DMAgentic.ArmyPanel@datamatics.com